Capitolul 3

Bazele logice ale calculatoarelor

Conf. dr. ing. Marius ROGOBETE

Introducere

- Caracteristica comună cea mai importantă a tuturor generaţiilor de calculatoare numerice realizate până în prezent și anume natura discretă a operaţiilor efectuate.
- Teoretic și practic s-a impus utilizarea dispozitivelor care codifică informaţia în două stări stabile, rezultând efectuarea calculelor în sistem binar.
- Suportul teoretic al acestuia este algebra logică (booleană).
- Analiza şi sinteza circuitelor de comutație aferente calculatoarelor numerice utilizează algebra booleană ca principal instrument matematic

Funcții logice. Tabele de adevăr.

- Funcţia logică conține un număr variabil de termeni.
- Numărul maxim de valori ce vor fi procesate de funcție este egal cu $\boldsymbol{2^i}$ (unde *i* este numărul de variabile ale funcției).
- În aparatura digitală valorile logice “0” şi “1” ale variabilelor funcţiei sunt reprezentate prin două nivele de tensiune diferite.
- Expresiile booleene sau funcţiile logice pot fi reprezentate în mai multe moduri ce vor fi exemplificate pe o funcție oarecare *f*.

Funcții logice. Tabele de adevăr.

- Tabela de adevăr este cea mai simplă reprezentare a unei funcții booleene. Aceasta cuprinde toate combinațiile posibile ale valorilor variabilelor de intrare şi afișează în dreptul fiecăreia, valoarea corespunzătoare, procesată la ieşire pentru funcţia *f*. Cu alte cuvinte, tabelul de adevăr afișează ieşirile pentru toate combinațiile posibile de valori de intrare.

- *Exemplu*: Tabela de adevăr pentru o funcție *f*(*A, B, C*) oarecare de trei variabile poate fi:

A	B	C	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Funcții logice. Tabele de adevăr.

Forma canonică normal disjunctivă

- Este o expresie care separă operatorii, fiind una dintre formele de reprezentare des întâlnite. Expresia constă din variabile conectate printr-un operator AND rezultând termeni conectaţi cu operatori OR. Această reprezentara poartă denumirea de sumă de produse sau formă canonică normal disjunctivă (f.c.n.d.).
- Fiecare operaţie AND poate fi privită ca un produs booleană, termenul obţinut din variabile conectate de operatori AND fiind un termen-produs. Operatorul OR este asimilat ca o însumare booleană, iar expresia cu termeni produs conectaţi de operatori OR fiind o expresie sumă-de-produse sau forma canonică normal disjunctivă.
- În exemplul următor, expresia funcției este o sumă de produse completă pentru o funcţie de trei variabile :

$$f(A,B,C) = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C + AB\bar{C} + ABC$$

Notând $\overline{A}\overline{B}\overline{C}$ cu P_0 , $\overline{A}\overline{B}C$ cu P_1 , etc., forma canonică normal disjunctivă se poate rescrie astfel:

$$f(A, B, C)= P_0 +P_1+ P_2 +P_3 +P_4 +P_5+P_6+P_7.$$

Funcții logice. Tabele de adevăr.

Forma canonică normal conjunctivă

$$f(A,B,C)=(A+B+C)(A+B+\overline{C})(A+\overline{B}+C)(A+\overline{B}+\overline{C})\cdot$$
$$\cdot(\overline{A}+B+C)(\overline{A}+B+\overline{C})(\overline{A}+B+C)(\overline{A}+\overline{B}+\overline{C}).$$

- Forma canonică normal conjuctivă (f.c.n.c.) este o altă modalitate de exprimare a funcţiilor. Aceasta se obține din operatori AND ce conectează termeni legaţi prin operatori OR. Pentru o funcție logică de trei variabile, forma canonică normal conjunctivă completă se scrie astfel:

Notând S_0 , S_1 etc, funcția se poate rescrie:

$$f(A, B, C)= S_0 S_1 S_2 S_3 S_4 S_5 S_6 S_7.$$

Funcții logice. Tabele de adevăr.

Diagrame Veitch-Karnaugh

- O reprezentare grafică a formelor canonice este dată de diagramele Veitch-Karnaugh. Aceasta constă dintr-o matrice, unde fiecărui element îi corespunde un termen produs canonic.
- Caracteristic pentru diagramele Veitch-Karnaugh este că orice element diferă de elementul său adiacent printr-o singură variabilă. Ca exemplu sunt reprezentate două diagrame Veitch-Karnaugh de trei şi patru variabile, rezultând astfel opt, respectiv şaisprezece combinații, fiecăreia dintre aceste combinații fiindui alocată câte un element din diagramă.

	$\bar{A}\bar{B}$ 0 0	$\bar{A}B$ 01	AB 11	$A\bar{B}$ 10
$\bar{C}$ 0	P_0	P_2	P_6	P_4
C 1	P_1	P_3	P_7	P_5

Figura 1
Diagrama V-K
pentru 3 variabile de intrare

	$\bar{A}\bar{B}$ 0 0	$\bar{A}B$ 01	AB 11	$A\bar{B}$ 10
$\bar{C}\bar{D}$ 00	P_0	P_4	P_{12}	P_8
$\bar{C}D$ 01	P_1	P_5	P_{13}	P_9
CD 11	P_3	P_7	P_{15}	P_{11}
CD 10	P_2	P_6	P_{14}	P_{10}

Figura 2
Diagrama V-K
pentru 4 variabile de intrare

Funcții logice. Tabele de adevăr.

Forma elementară

- Termenii formelor elementare nu conţin toate variabilele de intrare, spre deosebire de formele canonice prezentate anterior. Pornind de la forma de reprezentare canonică putem ajunge la una elementară prin operaţia numită minimizare.
- Exprimarea unei funcții prin forme elementare oferă avantaje faţă de formele canonice în primul rând la implementarea funcției, deoarece numărul de circuite şi componente electronice implicat este minimizat.
- Exemplu de scriere a unei funcții sub formă elementară:

$$f(A,B,C) = \overline{A}B + \overline{B}\,\overline{C}$$

Funcții logice. Tabele de adevăr.

Minimizarea funcțiilor logice

- *Tehnica minimizării permite exprimarea funcției printr-o formă elementară prin transformarea intr-o formă canonică, eliminând variabilele de intrare neutilizate din termenii funcției. Utilizarea expresiei elementare la implementare va costa mai puţin și/sau va opera mai rapid față de implementarea expresiei iniţiale.*

- *Printre cele mai răspândite metode de minimizare este utilizarea diagramele Veitch-Karnaugh. Prin această metodă se face o simplă identificarea vizuală a termenilor care pot fi combinaţi .*

Funcții logice. Tabele de adevăr.

Minimizarea funcțiilor logice

Tehnica minimizării cu ajutorul diagramelor Veitch-Karnaugh

Avem dată definiția funcţiei exprimată ca o sumă de produse;

I. Este dată definiția funcţiei exprimată ca o sumă de produse;

II. Elementele din diagrama Veitch-Karnaugh ce corespund termenilor din expresie sunt marcate cu 1; celelate elemente rămase pot fi marcate cu zerouri pentru a indica faptul că funcția va fi *0* în aceste situaţii, sau vor rămâne necompletate.

III. Se face gruparea suprafeţelor valide de valoare *1*, formate din elementele adiacente pe orizontală sau verticală (suprafeţele pot conţine un număr de elemente egal cu puteri ale lui 2).

IV. Elementele de-a lungul unei laturi sunt considerate adiacente inclusiv cu cele de pe latura opusă (sus şi jos sau stânga şi dreapta), întrucât ele corespund termenilor cu o singură variabilă diferită.

V. Suprafeţele maximale corespund termenilor elementari, iar reprezentarea grafică este ilustrarea teoremei:

$$A \cdot B + A \cdot \overline{B} = A$$

VI. Forma elementară se obține ca o sumă de produse pentru termenii elementari rezultaţi în etapa V.

Funcții logice. Tabele de adevăr.

Minimizarea funcțiilor logice

Exemplu: Să se minimizeze funcția

$$f = P_0+P_2+P_5+P_7+P_8+P_9+P_{10}+P_{11}+P_{12}+P_{14}$$

folosind diagrama V-K .

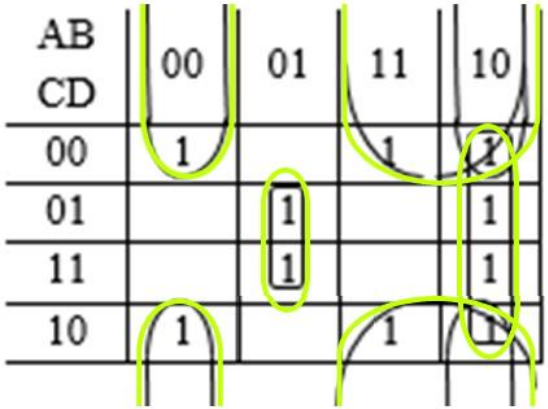
AB / CD	00	01	11	10
00	1		1	1
01		1		1
11		1		1
10	1		1	1

Funcții logice. Tabele de adevăr.

Minimizarea funcțiilor logice

REZOLVARE:

- $f = P_0+P_2+P_5+P_7+P_8+P_9+P_{10}+P_{11}+P_{12}+P_{14}$
- Pentru construirea diagramei Karnaugh se poate porni şi de la f.c.n.c., caz în care suprafeţele maximale vor fi date de celulele adiacente conţinând 0 logic.



- Se preferă, totuşi, lucrul cu f.c.n.d. care are avantajul, pe lângă comoditatea oferită de lucrul cu expresii algebrice care conţin sume de produse şi pe acela al implementării cu porţi tip **NAND**, mai răspândite şi mai avantajoase tehnologic

$$f = \overline{A}\,\overline{B}\,\overline{D} + \overline{A}BD + A\overline{D} + A\overline{B}$$

Funcții logice. Tabele de adevăr.

Minimizarea funcțiilor logice

REZOLVARE:

- $f = P_0+P_2+P_5+P_7+P_8+P_9+P_{10}+P_{11}+P_{12}+P_{14}$

AB+Ab=A

- F=[abcd+abCd] +

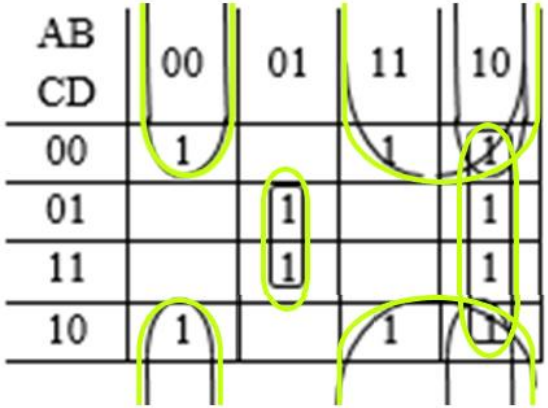
[aBcD+aBCD] +

[ABcd+Abcd+ABCd+AbCd]+

[Abcd+AbCD+AbcD+AbCd]

Abd+aBD + [Acd+ACd] + [Abc+AbC]

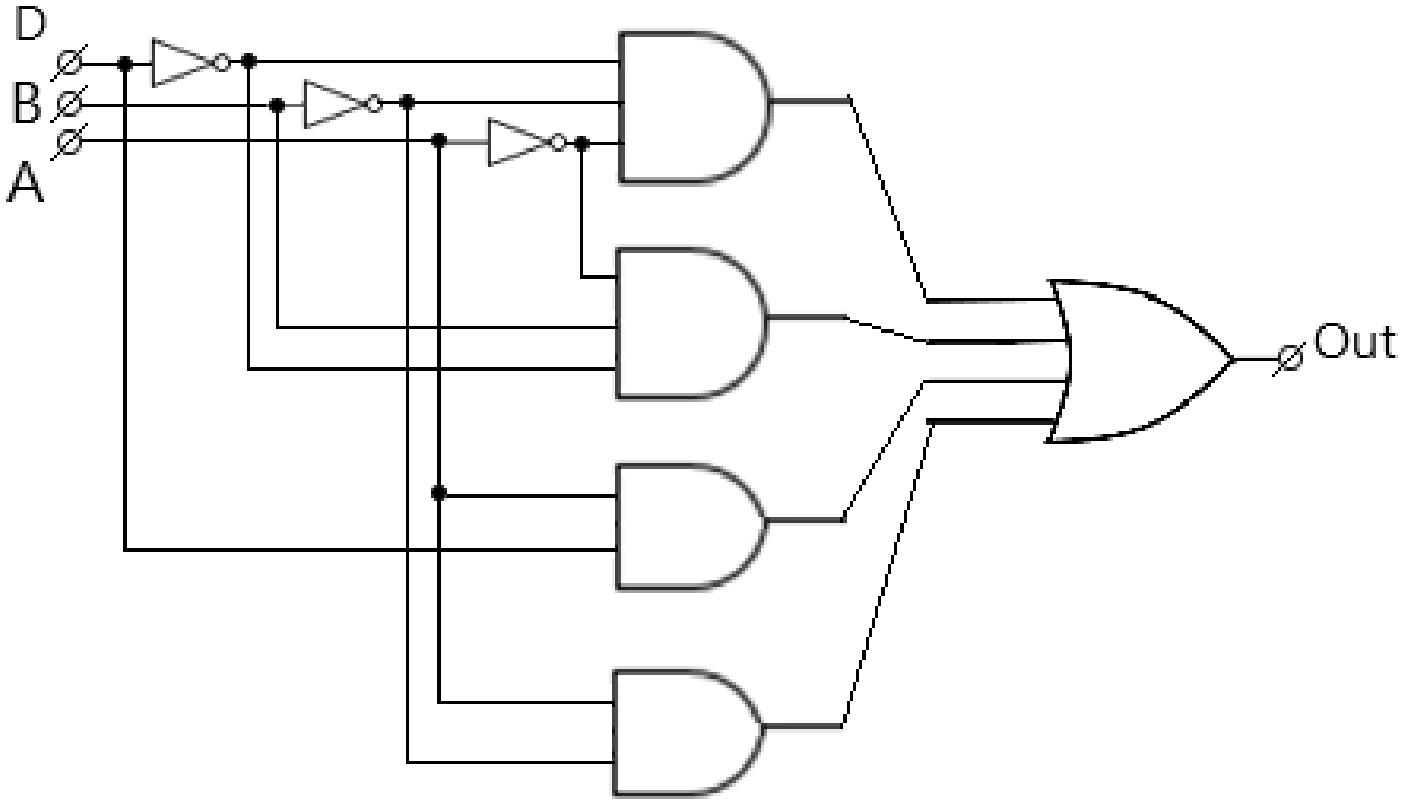
Abd+aBD +Ad + Ab



$$f = \overline{A}\,\overline{B}\,\overline{D} + \overline{A}BD + A\overline{D} + A\overline{B}$$

Funcții logice. Tabele de adevăr.

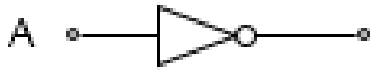
$$f = \overline{ABD} + \overline{A}BD + A\overline{D} + A\overline{B}$$



Porți logice

- Tensiunile porţilor logice produc un **nivel logic** de tensiune **"înaltă"** (**HIGH**) şi respectiv un nivel logic de tensiune **"joasă"** (**LOW**). Algebra booleană foloseşte trei operatori fundamentali cu care sunt definite toate funcţiile logice îndeplinite de porţile logice. Toate funcţiile care se obţin cu ajutorul acestor operatori sunt implementate de circuite numite porţi logice. Acestea sunt:
  - **NOT**
  - **AND**
  - **OR**

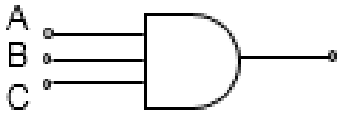
Poarta NOT



$$f = \overline{A}$$

A	f
0	1
1	0

Poarta AND



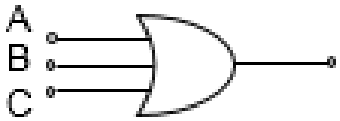
$$f = A \cdot B \cdot C$$

A	B	C	f
0	0	0	0
0	0	1	0
0	1	1	0
0	1	0	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

- Când toate intrările sunt SUS ieșirea este SUS.
- Când cel puțin o intrae este JOS ieșirea este JOS.

Porți logice

Poarta OR

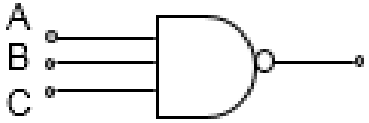


$$f = A + B + C$$

A	B	C	f
0	0	0	0
0	0	1	1
0	1	1	1
0	1	0	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Pentru orice intrare SUS ieșirea va fi SUS.

Poarta NAND



$$f = \overline{A \cdot B \cdot C}$$

A	B	C	f
0	0	0	1
0	0	1	1
0	1	0	1
1	0	0	1
0	1	1	1
1	0	1	1
1	1	0	1
1	1	1	0

Porți logice

Poarta NOR

$$f = \overline{A + B + C}$$

A	B	C	f
0	0	0	1
0	0	1	0
0	1	0	0
1	0	0	0
0	1	1	0
1	0	1	0
1	1	0	0
1	1	1	0

Poarta XOR

$$f = A \oplus B$$

A	B	f
0	0	0
0	1	1
1	0	1
1	1	0

Aplicații ale algebrei booleene

1. Să se minimizeze funcția

$$f = P_1+P_4+P_5+P_6 +P_7 +P_{12} +P_{14}$$

folosind diagrama V-K .

2. Sa se faca schema cu porti logice rezultata din minimizare.

Aplicații ale algebrei booleene

Diagrama V-K pentru funcția f = $P_1+P_4+P_5+P_6+P_7+P_{12}+P_{14}$:

P_n		$\overline{A}\,\overline{B}$	$\overline{A}B$	AB	$A\overline{B}$
		00	**01**	**11**	**10**
$\overline{C}\,\overline{D}$	**00**	*n=0*	*n=4*	*n=12*	*n=8*
$\overline{C}D$	**01**	*n=1*	*5*	*n=13*	*n=3*
CD	**11**	*n=3*	*n=7*	*n=15*	*n=11*
$C\overline{D}$	**10**	*n=2*	*n=6*	*n=14*	*n=10*

Aplicații ale algebrei booleene

Funcția f = $P_1+P_4+P_5+P_6+P_7+P_{12}+P_{14}$ minimizată folosind diagrama V-K:

		$\overline{A}\,\overline{B}$	$\overline{A}B$	AB	$A\overline{B}$
		00	**01**	**11**	**10**
$\overline{C}\,\overline{D}$	**00**	*0*	*4*	*12*	*8*
$\overline{C}D$	**01**	*1*	*5*	*13*	*3*
CD	**11**	*3*	*7*	*15*	*11*
$C\overline{D}$	**10**	*2*	*6*	*14*	*10*

$$f = \overline{A}\,\overline{B}\bar{C}D + \overline{A}B\bar{C}D +$$
$$+\overline{A}B\bar{C}\overline{D} + \overline{A}B\bar{C}D + \overline{A}BCD + \overline{A}BC\overline{D} +$$
$$+\overline{A}B\bar{C}\overline{D} + AB\bar{C}\overline{D} + \overline{A}BC\overline{D} + ABC\overline{D}$$

Aplicații ale algebrei booleene

Funcția f = $P_1+P_4+P_5+P_6+P_7+P_{12}+P_{14}$ minimizată folosind diagrama V-K:

		$\overline{A}\overline{B}$	$\overline{A}B$	AB	$A\overline{B}$
		00	**01**	**11**	**10**
$\overline{C}\overline{D}$	**00**	*0*	*4*	*12*	*8*
$\overline{C}D$	**01**	*1*	*5*	*13*	*3*
CD	**11**	*3*	*7*	*15*	*11*
$C\overline{D}$	**10**	*2*	*6*	*14*	*10*

$$f = \overline{A}\overline{B}\bar{C}D + \overline{A}B + B\overline{D}$$

Aplicații ale algebrei booleene

- C++:

```
.................
    // We need some TRUE/FALSE masks to convert CCA_BOOL values to something
    // that can be "anded" with the db field masks.
    int BOOL_TRUE_MASK = 0xFFFFFFFF;
    int BOOL_FALSE_MASK = 0x00000000;
    int BOOL_MASK;

    // Set the last connect time.
    m_DbInMemory.device.tlast[deviceIndex] = theDevice->GetLastConnectTime();

    // Set the in service flag.  In the database and on the display it is
    // Not In Service so it will be the opposite of the inservice? return.
    if ( !theDevice->IsInService() )
                BOOL_MASK = BOOL_TRUE_MASK;
   else
                BOOL_MASK = BOOL_FALSE_MASK;

    m_DbInMemory.device.service[deviceIndex] = ( NIS_DEVICE & BOOL_MASK );
....................
```

Aplicații ale algebrei booleene

- Java (1):

Bit flags, commonly referred to as Bit fields are an efficient way of storing several related boolean values in a single primitive type. Internally represented in binary, you can decide on how large the storage type needs to be - for example, a Java integer will provide you with space for 31 flags. Being a 32 bit type you would assume to have access to this many flags, however, due to it being a signed value the most significant bit is reserved to store its +/- sign.

Flags are typically stored as public constants within the related class, for example, the Pattern class includes the flags 'CASE_INSENSITIVE' and 'MULTILINE' to alter match criteria. Setting the values as incremental powers of 2 allows the user to compose multiple constants together using the | (OR) operator to achieve the desired flag combination. The implementer is then able to base the methods execution on the passed in flags by checking for presence of the constant value using the & (AND) operator as a bit mask. This idea can be more meaningfully described using the example below

Aplicații ale algebrei booleene

- Java(2):

```
public static final int UPPERCASE = 1;  // 0001
public static final int REVERSE   = 2;  // 0010
public static final int FULL_STOP = 4;  // 0100
public static final int EMPHASISE = 8;  // 1000
public static final int ALL_OPTS  = 15; // 1111

public static String format(String value, int flags)
{
    if ((flags & UPPERCASE) == UPPERCASE) value = value.toUpperCase();

    if ((flags & REVERSE) == REVERSE) value = new
StringBuffer(value).reverse().toString();

    if ((flags & FULL_STOP) == FULL_STOP) value += ".";

    if ((flags & EMPHASISE) == EMPHASISE) value = "~*~ " + value + " ~*~";

    return value;
}
```

Algebra de comutație

Când vorbim despre algebra booleană sau algebra de comutație ne referim la o structură algebrică de tip:

***A**= (B, O)*

Unde:

***B**= {0, 1}* - este mulțimea constantelor de comutație,

***O** = {AND, OR, NOT}* - este mulțimea operatorilor de comutație, definiți astfel:

$$\forall x, y \in \mathbf{B} \qquad x \ AND \ y = \begin{cases} 1 \ dacă \ x = 1 \ și \ y = 1 \\ 0 \ dacă \ x = 0 \ și \ y = 0 \end{cases}$$

$$\forall x, y \in \mathbf{B} \qquad x \ OR \ y = \begin{cases} 1 \ dacă \ x = 1 \ sau \ y = 1 \\ 0 \ dacă \ x = 0 \ sau \ y = 0 \end{cases}$$

$$\forall x \in \mathbf{B} \qquad NOT \ x = \begin{cases} 1 \ dacă \ x = 0 \\ 0 \ dacă \ x = 1 \end{cases}$$

<u>Echivalențe ale operatorilor:</u>

- $\mathrm{AND} \equiv$ ȘI,$\cap$,$\cdot$ $(produs\ logic)$;
- $\mathrm{OR} \equiv$ SAU,$\cup$, $+(sumă\ logică)$;
- $\mathrm{NOT} \equiv$ NU, #, $(produs\ logic)$;

Algebra de comutație

O expresie de comutație este o combinație a unui număr finit de variabile de comutație, constante de comutație si operatori de comutație, definită astfel:

- orice constantă sau variabilă de comutație este o expresie de comutație;
- dacă ***t1*** si ***t2*** sunt expresii de comutație atunci ***t1•t2***, ***t1+t2*** și $\overline{t1}$ sunt expresii de comutație.

Evaluare a unei expresii de comutație se bazează pe următoarele reguli:

- dacă există paranteze, evaluarea se face din interior spre exterior;
- ordinea descrescătoare a priorităților operatorilor este: ***NOT, AND, OR***;
- pentru priorități egale evaluarea se face de la stânga la dreapta.

Funcții de comutație

- Funcția f : $\mathbf{B}^n$ -> **B** este o funcție de comutație de n variabile.
- Funcția *complementară* a unei funcții f, notată cu $\bar{f}$ este acea funcție care are valoarea *1 / 0* pentru toate combinațiile de valori ale variabilelor pentru care funcția f ia valoarea *0 / 1*.
- Specificarea unei funcții de comutație de n variabile se poate face în principal prin tabela de adevăr sau prin expresia de comutație.
- Tabela de adevăr prezintă valorile funcției pentru toate cele 2^n combinații de valori ale variabilelor.
- *Exemplu.* Se considera funcția f : $\mathbf{B}^3$ -> **B** , data prin tabela de adevăr:

x	y	z	f
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

Algebra de comutație

Pe baza tabelei de adevăr se obține expresia de comutație a funcției în umătorul mod:

- expresia se scrie sub forma unei sume de produse, câte un termen al sumei pentru fiecare valoare 1 a funcției (în cazul nostru expresia conține trei termeni);
- pentru fiecare termen se scrie un produs al tuturor variabilelor funcției, fiecare variabilă reprezentandu-se în formă directă dacă variabila are valoarea 1 în combinația respectivă de valori din tabelă sau în formă complementată (negată) dacă variabila are valoarea 0. Se obține:

$$f(x,y,z)=\overline{x}\cdot\overline{y}\cdot\overline{z}+\overline{x}\cdot\overline{y}\cdot z+x\cdot\overline{y}\cdot z$$

Această sumă de produse, în care fiecare produs conține toate variabilele atât în formă directa cât și negată se numește *forma canonică sumă de mintermeni* (un mintermen fiind un produs al tuturor variabilelor în formă directă sau negată). Ne interesează expresia minimă (suma de produse) obținută aplicând proprietățile (de ex. $A\cdot B+A\cdot\overline{B}=A$) :

$$f(x,y,z)=\overline{x}\cdot\overline{y}\cdot\overline{z}+\overline{x}\cdot\overline{y}\cdot z+x\cdot\overline{y}\cdot z=\overline{x}\cdot\overline{y}\cdot\overline{z}+\overline{x}\cdot\overline{y}\cdot z+\overline{x}\cdot\overline{y}\cdot z+x\cdot\overline{y}\cdot z=$$

$$=\overline{x}\cdot\overline{y}\cdot(\overline{z}+z)+\overline{x}+x)\cdot\overline{y}\cdot z=\overline{x}\cdot\overline{y}\cdot 1+1\cdot\overline{y}\cdot z=\overline{x}\cdot\overline{y}+\overline{y}\cdot z.$$

Algebra de comutație

Pe baza tabelei de adevăr se obține expresia de comutație a funcției în umătorul mod:

- expresia se scrie sub forma unei sume de produse, câte un termen al sumei pentru fiecare valoare 1 a funcției (în cazul nostru expresia conține trei termeni);
- pentru fiecare termen se scrie un produs al tuturor variabilelor funcției, fiecare variabilă reprezentandu-se în formă directă dacă variabila are valoarea 1 în combinația respectivă de valori din tabelă sau în formă complementată (negată) dacă variabila are valoarea 0. Se obține:

$$f(x, y, z) = \overline{x} \cdot \overline{y} \cdot \overline{z} + \overline{x} \cdot \overline{y} \cdot z + x \cdot \overline{y} \cdot z$$

Această sumă de produse, în care fiecare produs conține toate variabilele atât în formă directa cât și negată se numește *forma canonică sumă de mintermeni* (un mintermen fiind un produs al tuturor variabilelor în formă directă sau negată). Ne interesează expresia minimă (suma de produse) obținută aplicând proprietățile de mai sus, pentru exemplul considerat fiind:

$$f(x, y, z) = \overline{x} \cdot \overline{y} \cdot \overline{z} + \overline{x} \cdot \overline{y} \cdot z + x \cdot \overline{y} \cdot z = \overline{x} \cdot \overline{y} \cdot \overline{z} + \overline{x} \cdot \overline{y} \cdot z + \overline{x} \cdot \overline{y} \cdot z + x \cdot \overline{y} \cdot z =$$

$$= \overline{x} \cdot \overline{y} \cdot (\overline{z} + z) + (\overline{x} + x) \cdot \overline{y} \cdot z = \overline{x} \cdot \overline{y} \cdot 1 + 1 \cdot \overline{y} \cdot z = \overline{x} \cdot \overline{y} + \overline{y} \cdot z.$$

Algebra de comutație

Observatii.

1. Orice funcție de comutație de *n* variabile se poate reprezenta în mod unic printr-o expresie de formă canonică sumă de mintermeni;

2. Pentru *n* variabile exista 2^n mintermeni;

3. Pentru o combinație de valori ale celor *n* variabile un singur mintermen are valoarea 1, toti ceilalti mintermeni au valoarea 0;

4. Produsul a doi mintermeni diferiți este 1.

Logică și circuite logice combinaționale (CLC)

Stabilirea structurii circuitului se face prin analiza ce presupune parcurgerea următoarelor etape:

- definirea funcţiilor logice;
- minimizarea acestora;
- obţinerea schemei circuitului.

Tipurile reprezentative de *CLC* din structura unui calculator numeric sunt:

1. *convertoarele de cod,*
2. *codificatoarele şi decodificatoarele,*
3. *multiplexoarele şi demultiplexoarele,*
4. *comparatoarele,*
5. *detectoarele şi generatoarele de paritate, ariile logice programabile, memoriile şi circuitele aritmetice.*

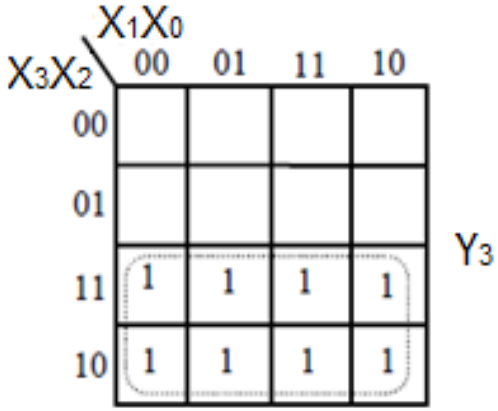
1. Convertoare de cod (1)

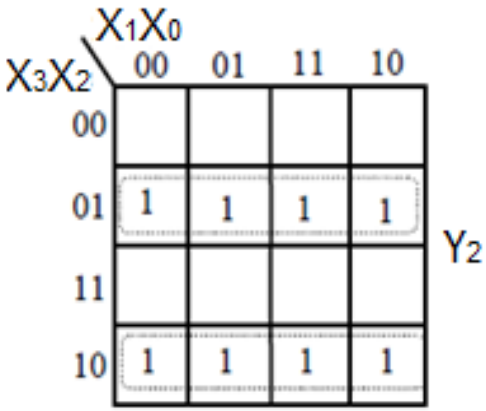
CLC-ul care permite trecerea dintr-un cod binar în altul sunt convertoarele de cod. Sinteza unui asemenea CLC se va exemplifica pentru un convertor *din cod binar în cod Gray*. În figură se prezintă elementele aferente sintezei acestui tip de convertor, în care X_3 X_2 X_1 X_0 reprezintă cuvântul binar aplicat la intrare, iar Y_3 Y_2 Y_1 Y_0 cuvântul binar obţinut la ieşire.

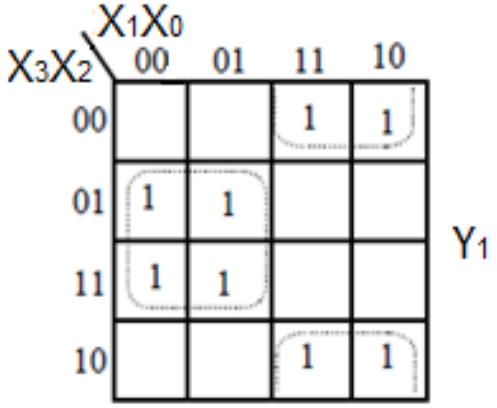
X_3	X_2	X_1	X_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

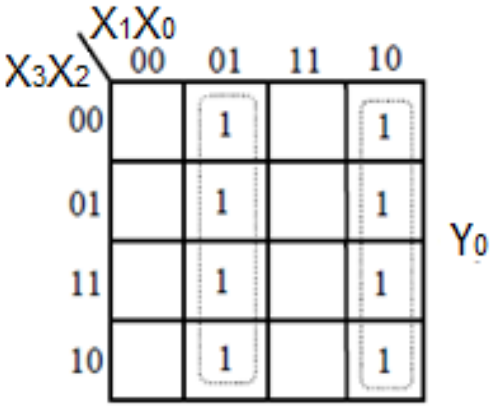
Convertor de cod natural- Gray: tabela de corespondență.

1. Convertoare de cod (2)



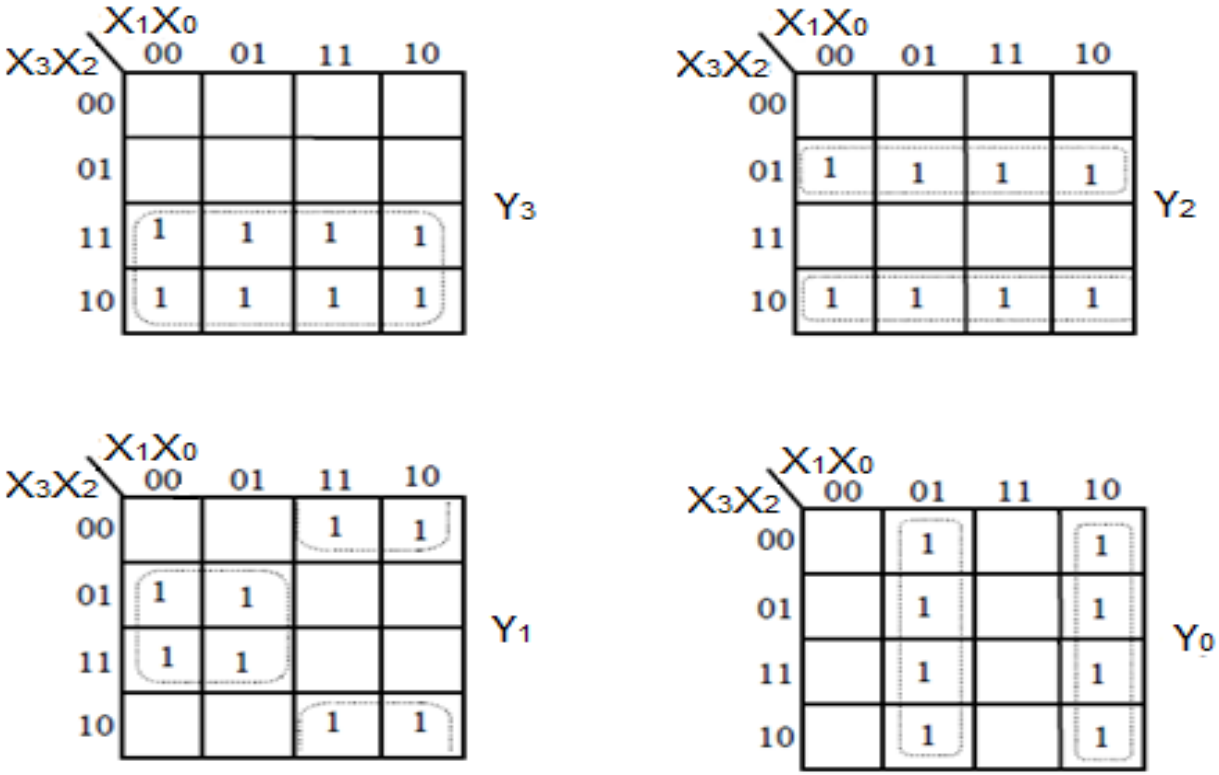






X_3	X_2	X_1	X_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

1. Convertoare de cod (3)



Convertor de cod natural- Gray: diagramele Karnaugh asociate

După reducerile în diagramele Karnaugh rezultă:

$$Y_3 = X_3$$

$$Y_2 = \overline{X_2}X_3 + \overline{X_3}X_2 = X_2 \oplus X_3$$

$$Y_1 = \overline{X_1}X_2 + \overline{X_2}X_1 = X_1 \oplus X_2$$

$$Y_0 = \overline{X_1}X_0 + \overline{X_0}X_1 = X_1 \oplus X_0$$

2. Codificatoare și decodificatoare

CLC la care activarea unei intrări, dintr-un grup de *m,* conduce la apariţia unui cuvânt de cod la ieşire format din *n* biţi se numesc c*odificatoare.* În figură se prezintă elementele unui codificator cu *m=3* intrări şi *n=2* ieșiri.

X_0	X_1	X_2	Y_0	Y_1
0	0	0	0	0
1	0	0	0	1
0	1	0	1	0
0	0	1	1	1

$$Y_0 = \overline{X_0} \cdot (X_1 \oplus X_2)$$
$$Y_1 = \overline{X_1} \cdot (X_0 \oplus X_2)$$

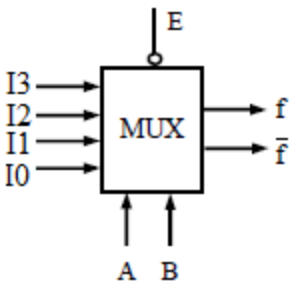
Codificator cu m=3 şi n=2 (schema bloc, tabela de adevăr, funcții logice)

Decodificatoarele sunt acele *CLC* care activează una sau mai multe ieşiri funcție de cuvântul de cod aplicat la intrare și este necesară în aplicaţii care adresează memoria, în afişarea numerică, multiplexarea datelor etc**.**

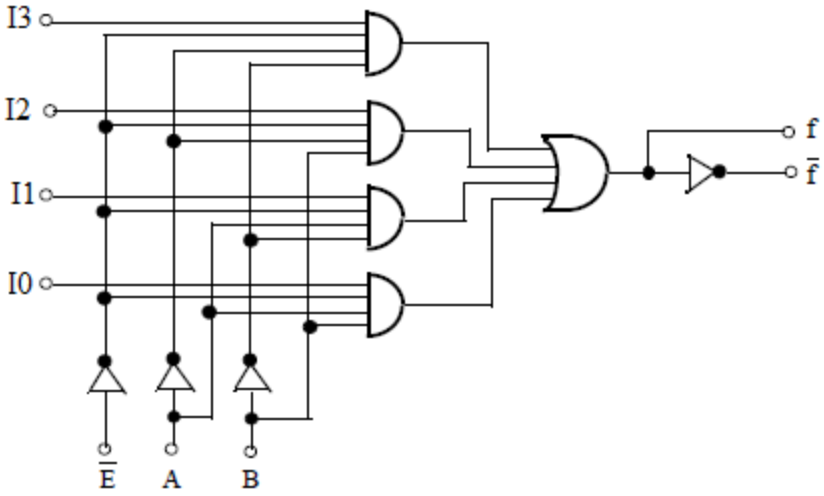
3. Multiplexoare și demultiplexoare

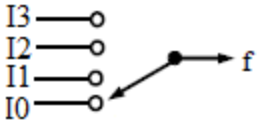
Multiplexoarele (MUX) sunt *CLC* care fac transferul datelor de la una din intrările selectate de o adresă (cuvânt de selecţie) către o ieșire unică. Funcţional *MUX* pot fi asimilate cu o reţea de comutatoare comandate. *Multiplexoarele* pot fi analogice sau numerice, ultimele fiind specifice CN.

E	B	A	f = Canal
0	0	0	I3
0	0	1	I2
0	1	0	I1
0	1	1	I0
1	*	*	-



Exemplu: sinteza unui MUX 4:1 numeric şi implementarea cu porţi logice:



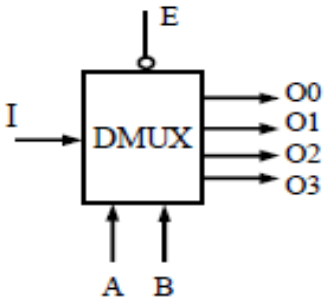


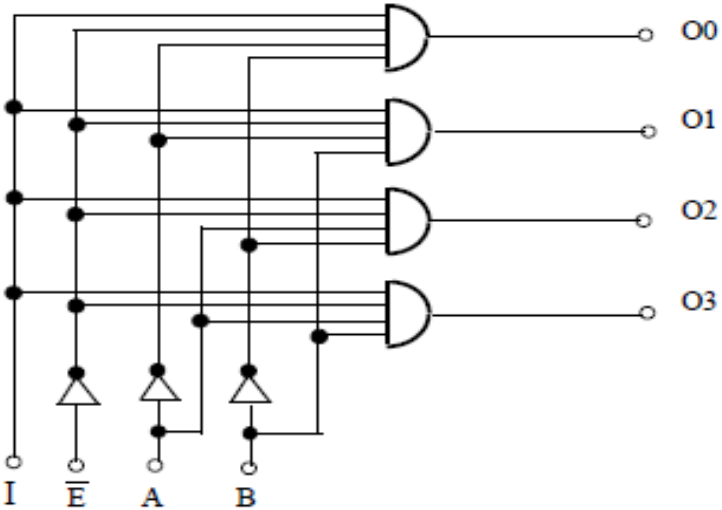
$$f = \overline{E} \cdot (\overline{B} \cdot \overline{A} \cdot I3 + \overline{B} \cdot A \cdot I2 + B \cdot \overline{A} \cdot I1 + B \cdot A \cdot I0)$$

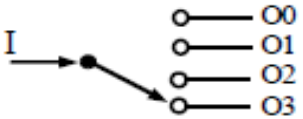
3. Multiplexoare și demultiplexoare

Exemplu: Demultiplexor 1:4:

E	B	A	O0	O1	O2	O3
0	0	0	I			
0	0	1		I		
0	1	0			I	
0	1	1				I
1	*	*	-	-	-	-







$O0 = \overline{E} \cdot \overline{B} \cdot \overline{A} \cdot I$ $\quad O2 = \overline{E} \cdot B \cdot \overline{A} \cdot I$

$O1 = \overline{E} \cdot \overline{B} \cdot A \cdot I$ $\quad O3 = \overline{E} \cdot B \cdot A \cdot I$

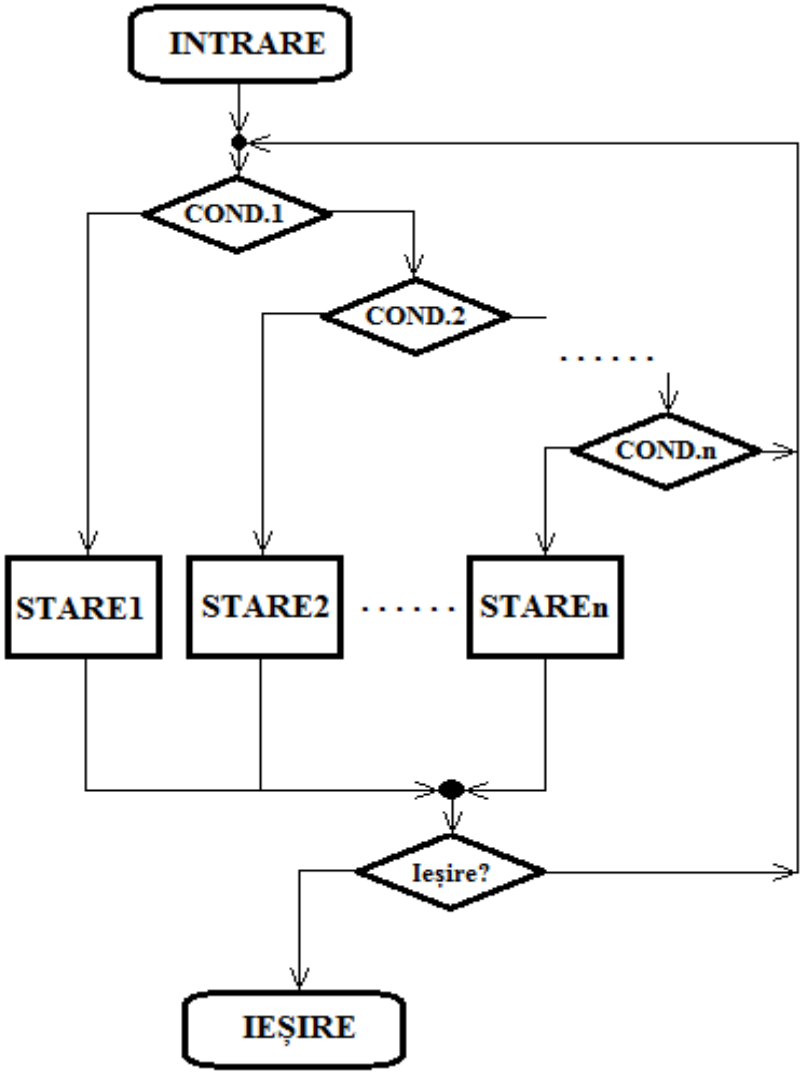
Maşini cu număr finit de stări

Un **automat finit** (AF) sau o **maşină cu stări finite** este un model de comportament compus din *stări*, *tranziţii* şi *acţiuni*.

- O stare reține comportamentul compus din *alte stări*, *tranziţii* şi *acţiuni.* Starea stochează informaţii despre trecut, adică reflectă schimbările intrării de la iniţializarea sistemului până în momentul de faţă.
- Tranziţia indică o schimbare de stare şi este descrisă de o condiţie care este nevoie să fie îndeplinită pentru a declanşa tranziţia.
- Acţiunea este o descriere a unei activităţi ce urmează a fi executată la un anumit moment. Există câteva tipuri de acţiuni:
    - Acţiune *de intrare* executată la intrarea într-o stare.
    - Acţiune *de ieşire* executată la ieşirea dintr-o stare.
    - Acţiune *de intrare de date* executată în funcție de starea prezentă şi de datele de intrare.
    - Acţiune *de tranziţie* executată în momentul unei tranziţii.

Maşini cu număr finit de stări

Logica automatelor finite stabileşte că ieșirea şi starea următoare a unui automat finit este funcție de valoarea intrării şi de starea curentă.



Maşini cu număr finit de stări

Modelul matematic

În funcție de tip, există mai multe definiţii.

- Un automat finit **acceptor** este un cvintuplu <***Σ, S, s0, δ, F***>, unde:
  - ***Σ*** este alfabetul de intrare (o mulţime finită şi nevidă de simboluri).
  - ***S*** este o mulţime finită şi nevidă de stări.
  - ***s0*** este starea iniţială, element al lui ***S***. Într-un automat finit nedeterminist, ***s0*** este o mulţime de stări iniţiale.
  - ***δ*** este funcția de tranziţie a stării: ***δ*: *S* x *Σ* → *S***.
  - ***F*** este mulţimea stărilor finale, o submulţime (posibil vidă) a lui **S**.

- Un automat finit **transductor** este un sextuplu <***Σ, Γ, S, s0, δ, ω***>, unde:
  - ***Σ, S*** şi ***s0*** îşi păstrează semnificaţiile din definiţia automatului finit acceptor.
  - ***Γ*** este alfabetul de ieșire (o mulţime finită şi nevidă de simboluri).
  - ***δ*** este funcția de tranziţie a stării ***δ*: *S* x *Σ* → *S* x *Γ***.
  - ***ω*** este funcția de ieşire.

- Dacă funcția de ieşire este o funcţie de stare şi de alfabetul de intrare (***ω*: *S* x *Σ* → *Γ***), atunci această definiţie corespunde **modelului Mealy**.
- Dacă funcția de ieșire depinde doar de stare (***ω*: *S* → *Γ***), atunci această definiţie corespunde **modelului Moore**.